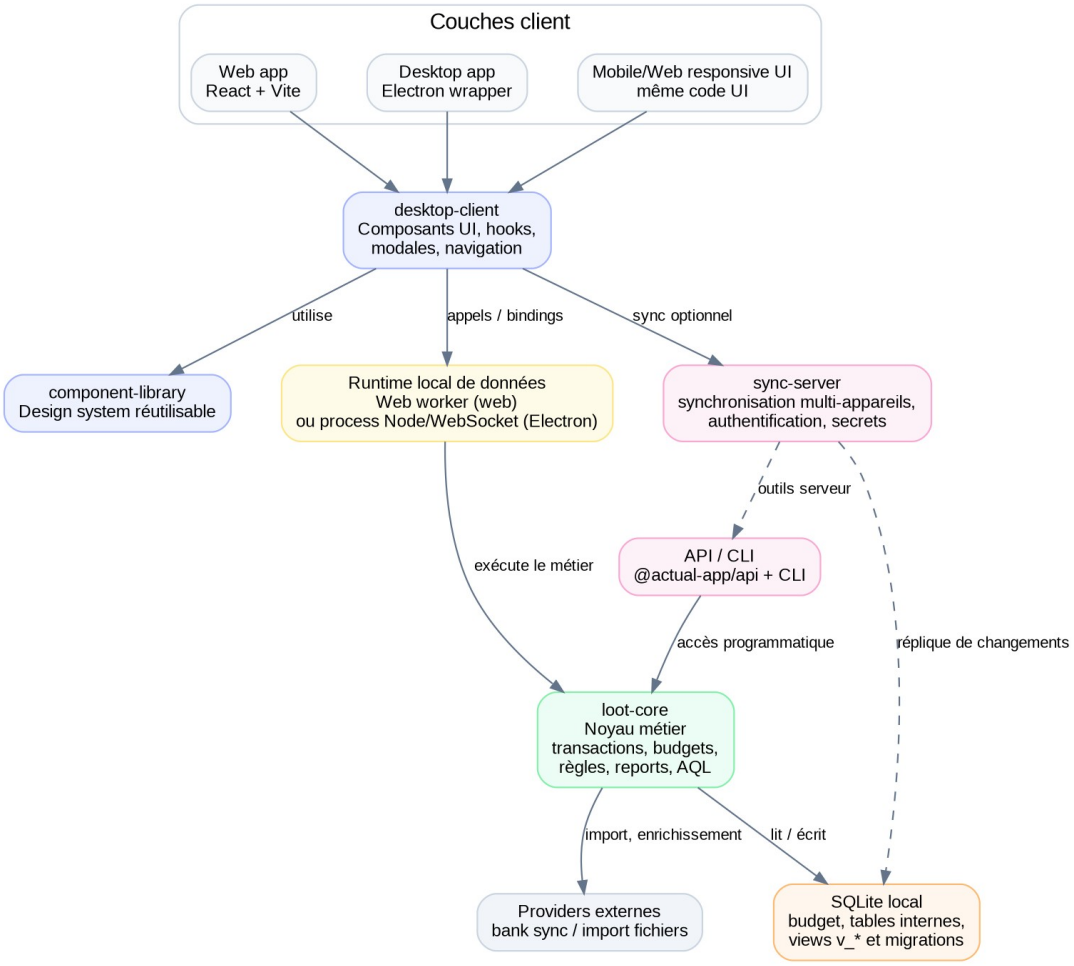
Actual
Dossier de documentation fonctionnelle et d'architecture

Version approfondie
Structurée pour prise en connaissance, analyse et évolution

Périmètre couvert
Architecture runtime, domaines fonctionnels, entités métier, gestion des comptes, cycle de vie des transactions, imports, règles, schedules, rapprochement, sync, rapports, API/CLI



Source de travail : export du dépôt Actual fourni le 15/04/2026

Sommaire

Mettez à jour le sommaire à l’ouverture du document si Word le demande.

1. Objet du livrable

Ce dossier vise à fournir une lecture structurée et exploitable du projet Actual avant évolution. Il combine une documentation fonctionnelle, une lecture d'architecture, une cartographie des classes fonctionnelles et des diagrammes de séquence couvrant les principaux parcours métier.

Le document est volontairement orienté compréhension du système : il ne décrit pas chaque écran à plat, mais relie les usages utilisateurs, les responsabilités métier et les composants techniques qui les portent.

2. Résumé exécutif

- Actual est une application de budget local-first : le budget vit en local sur l'appareil et peut être synchronisé vers un sync-server optionnel.
- Le front principal est un client React (desktop-client) ; la logique métier transverse réside dans loot-core.
- Le stockage local s'appuie sur SQLite ; le schéma réel résulte d'un modèle initial puis de migrations successives.
- Le runtime diffère selon le mode : web worker côté navigateur, processus Node enfant + WebSocket côté Electron.
- Les fonctions métier pivot sont : comptes, budget enveloppe, catégories, payees, transactions, transferts, règles, schedules, rapports, sync et accès programmatique via API/CLI.
- La transaction constitue l'objet central ; elle peut être saisie, importée, transformée par des règles, liée à un schedule, scindée en sous-transactions, rapprochée et synchronisée.

3. Vision produit et périmètre fonctionnel

Actual est un outil de gestion budgétaire personnelle orienté envelope budgeting. Le système distingue les comptes on-budget et off-budget, permet la tenue d'un registre de transactions, propose des catégories budgétaires avec report (rollover), automatise les imports au moyen de règles et fournit des rapports analytiques. Les données restent locales par défaut ; la synchronisation multi-appareils et le chiffrement de bout en bout sont optionnels.

Domaine	Finalité fonctionnelle	Objets centraux
Comptes	Représenter les sources et usages d'argent	Account, balances, reconciliation
Budget	Allouer l'argent disponible par enveloppes	BudgetMetaData, Category, MonthlySubCategoryBudget
Transactions	Tracer l'activité financière	Transaction, SubTransaction, cleared, accepted
Payees	Normaliser les contreparties et apprentissage	Payee, règles associées, catégorie par défaut

Règles	Automatiser classement et transformations	conditions, actions, ordre/spécificité
Schedules	Piloter les opérations récurrentes	ScheduledTransaction, posting, skip
Rapports	Analyser dépenses, cash flow, net worth	AQL, dashboards, custom reports
Sync / API	Multi-device et automatisation	sync-server, CLI, API

4. Architecture d'ensemble

L'architecture Actual sépare clairement la couche d'interface, le noyau métier et la persistance locale. Le sync-server n'est pas le backend métier principal de l'application : il sert d'orchestrateur de synchronisation et d'accès protégé au budget. La logique métier effective est portée par loot-core, exécuté localement selon un runtime adapté à la plateforme.

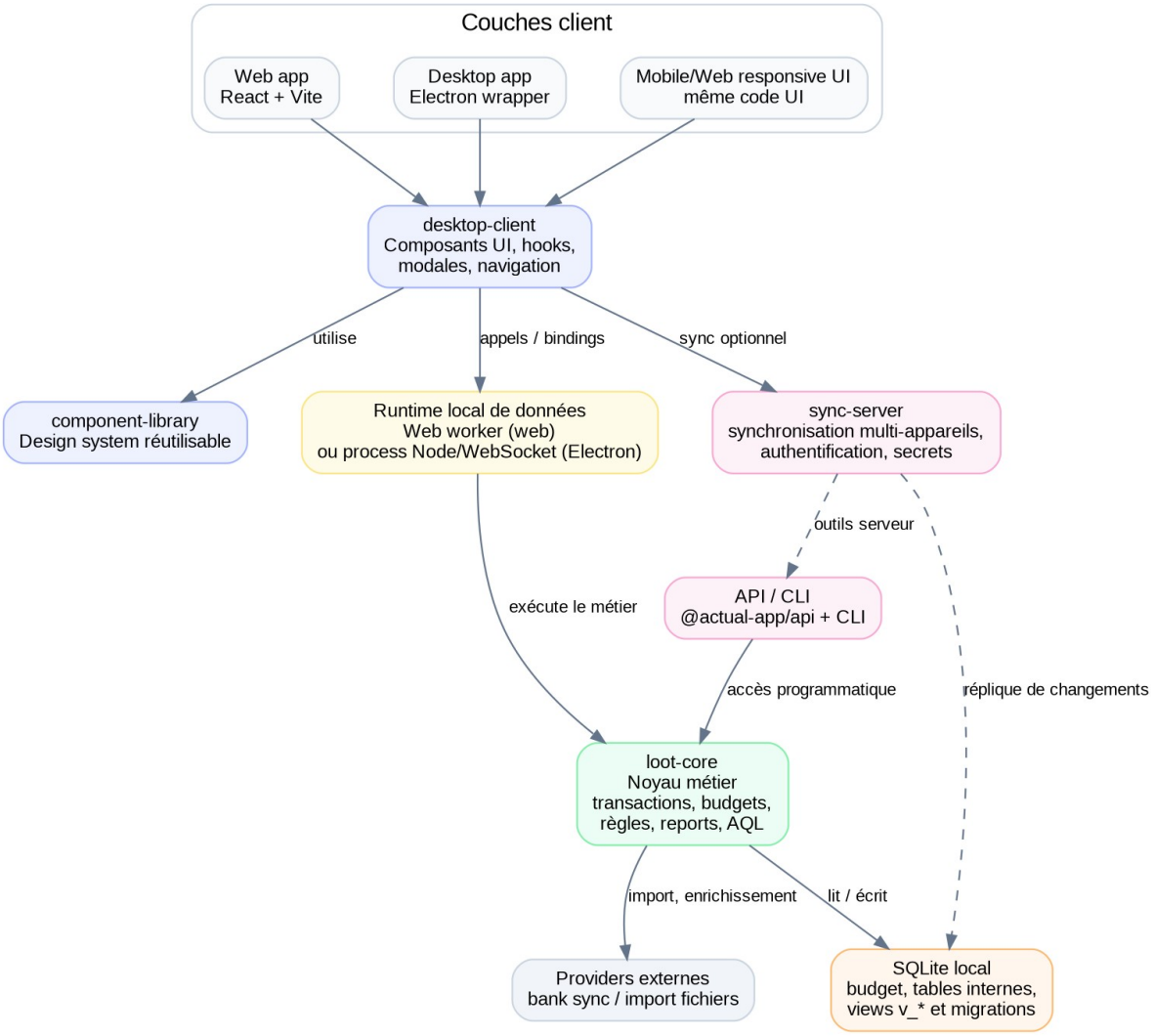


Figure 1 - Architecture d'ensemble du projet Actual

Lecture du schéma : le client React orchestre l'interface, s'appuie sur la component-library pour le rendu, délègue la logique métier à loot-core et persiste dans SQLite. Le sync-server intervient comme couche optionnelle de diffusion/réconciliation des changements entre appareils. L'API et la CLI s'appuient elles aussi sur les briques de cœur, ce qui renforce la cohérence entre usages UI et usages programmatiques.

4.1 Architecture runtime : web vs Electron

Le comportement runtime change selon le canal de déploiement. En web, le serveur local tourne dans un web worker afin d'isoler les opérations de base de données et préserver la réactivité du thread UI. En Electron, la logique de fond s'exécute dans un processus Node enfant relié à l'UI via WebSocket.

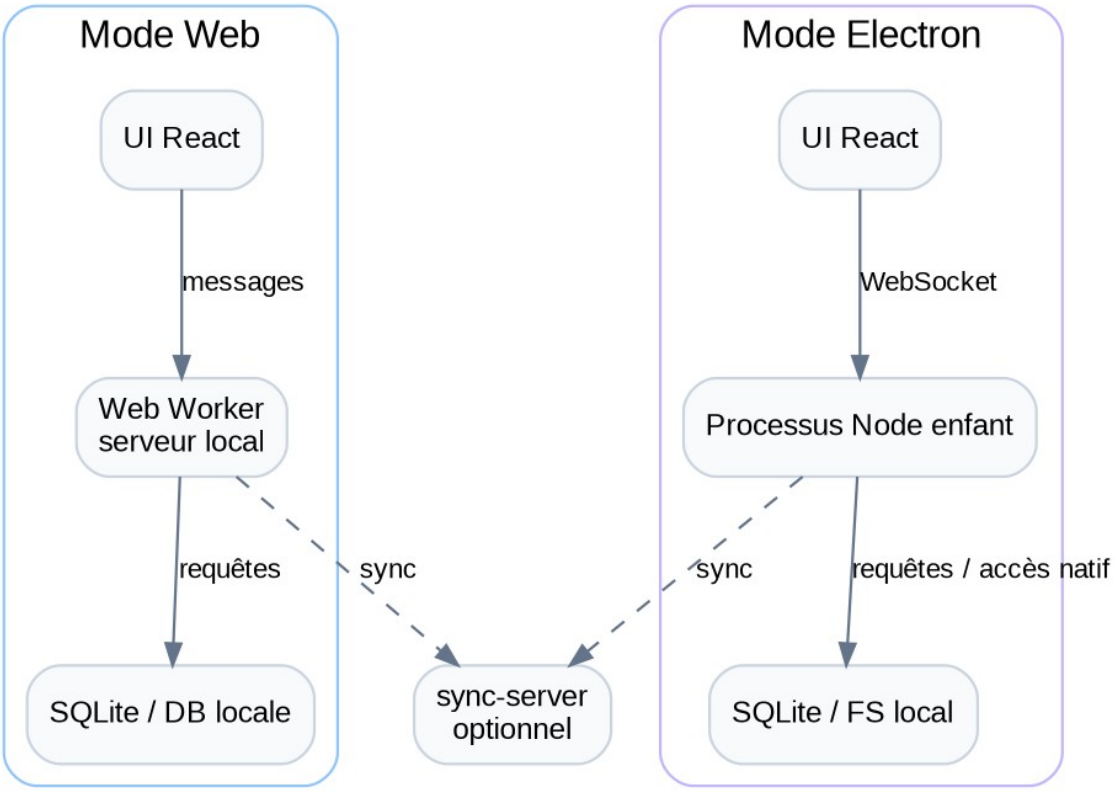


Figure 2 - Différence de runtime entre le mode web et le mode Electron

4.2 Structure de packages

Package	Rôle	Remarques
packages/loot-core	Noyau métier transversal	Transactions, budgets, règles, DB, AQL, logique partagée
packages/desktop-client	Client React web/desktop	Écrans, hooks, navigation,

		composants métier
packages/desktop-electron	Wrapper Electron	Fenêtrage, intégration OS, runtime desktop
packages/sync-server	Synchronisation serveur	Auth, sync multi-appareils, gestion serveur
packages/api	Accès programmatique	Intégrations et automatisation
packages/cli	Outillage en ligne de commande	Comptes, budgets, transactions, payees, tags, règles, schedules, query
packages/component-library	Bibliothèque UI	Design system et composants réutilisables
packages/docs	Documentation produit et contribution	Docs fonctionnelles, techniques, release notes

5. Modèle métier et classes fonctionnelles

Le projet manipule un nombre restreint d'objets métier structurants. Le schéma ci-dessous ne prétend pas refléter l'intégralité du modèle technique complet, mais il synthétise les classes fonctionnelles utiles pour raisonner sur l'évolution du produit.

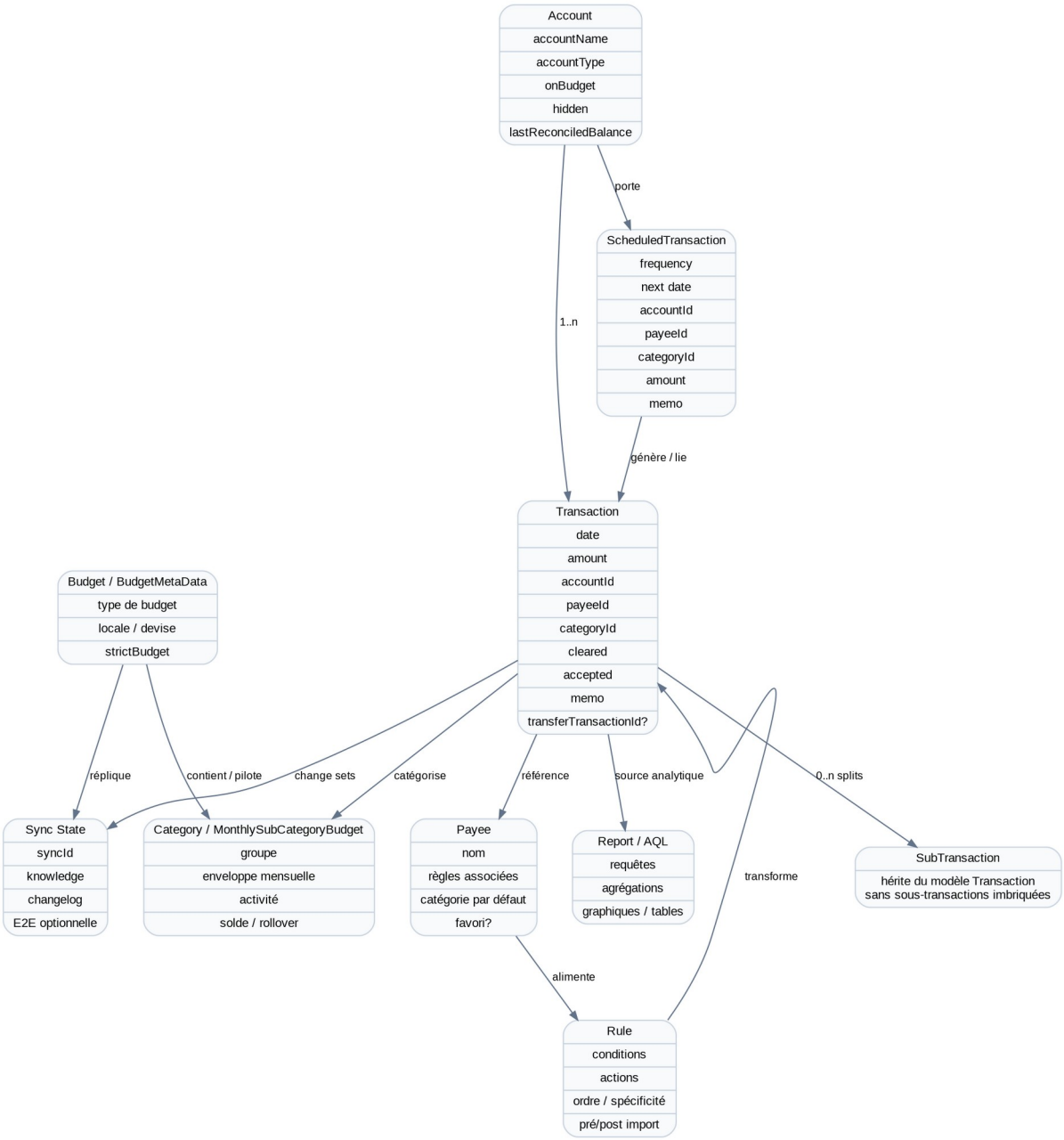


Figure 3 - Cartographie des classes fonctionnelles

Trois lectures sont particulièrement importantes. Premièrement, Budget et Category structurent l'allocation. Deuxièmement, Account et Transaction structurent le registre et les balances. Troisièmement, Rule, Payee et ScheduledTransaction sont des objets d'automatisation qui enrichissent ou génèrent des transactions.

5.1 Entités cœur

Entité	Champs clefs observés	Rôle fonctionnel
Account	accountName, accountType, onBudget, hidden,	Représente un compte financier et son rattachement

	lastReconciledBalance	budgétaire
Transaction	date, amount, accountId, payeeId, categoryId, cleared, accepted, memo	Événement financier atomique du registre
SubTransaction	variante de Transaction	Ventilation d'une transaction en plusieurs postes
ScheduledTransaction	frequency, date, amount, accountId, payeeId, categoryId	Modèle d'occurrence récurrente
BudgetMetaData	strictBudget, currencyISOSymbol, currencyLocale, budgetType, dateLocale	Paramétrage global du budget
MonthlySubCategoryBudget	categoryId, budgeted, overspendingHandling	Allocation mensuelle par sous-catégorie
FileMetaData	budgetDataVersion, currentKnowledge	Métadonnées de versionnement et de sync

5.2 Domaines fonctionnels

Comptes

- Création de comptes locaux ou liés à un provider de bank sync.
- Distinction on-budget / off-budget : seuls les comptes on-budget affectent le budget.
- Clôture de compte avec transfert du solde résiduel vers un autre compte.
- Historique, balances, cleared total, uncleared total, réconciliation.

Budget et catégories

- Approche envelope budgeting : on ne budgete que l'argent déjà disponible.
- Catégories regroupées en groupes ; allocations mensuelles et comportements de dépassement.
- Masquage, fusion, suppression et déplacement de soldes/transactions entre catégories.

Transactions

- Ajout, modification, suppression, déplacement, export CSV, merge, split.
- Gestion des flags cleared / reconciled / accepted selon les flux.
- Support des transferts via payees spéciaux et génération de transaction miroir.

Payees et règles

- Normalisation des noms importés, fusion et renommage de payees.
- Règles ordonnées : conditions + actions, la règle la plus spécifique s'exécute plus tard.
- Apprentissage implicite à partir des comportements de catégorisation/renommage.

Schedules

- Récurrences souples : mensuel, hebdo, jours spécifiques, one-shot, fin de validité.
- Prévisualisation, posting automatique ou manuel, skip de la prochaine occurrence.
- Lien entre transactions existantes et schedules.

Rapports et AQL

- Rapports natifs : net worth, cash flow, vues tableau / graphiques.
- Custom reports et moteur AQL pour requêtes analytiques.
- Les transactions alimentent les rapports via vues et agrégations.

6. Cycle de vie d'une transaction

La transaction est le centre de gravité fonctionnel du système. La majorité des parcours utilisateurs visent à produire, corriger, rapprocher, classer ou analyser des transactions. Le cycle de vie ci-dessous synthétise le traitement commun.

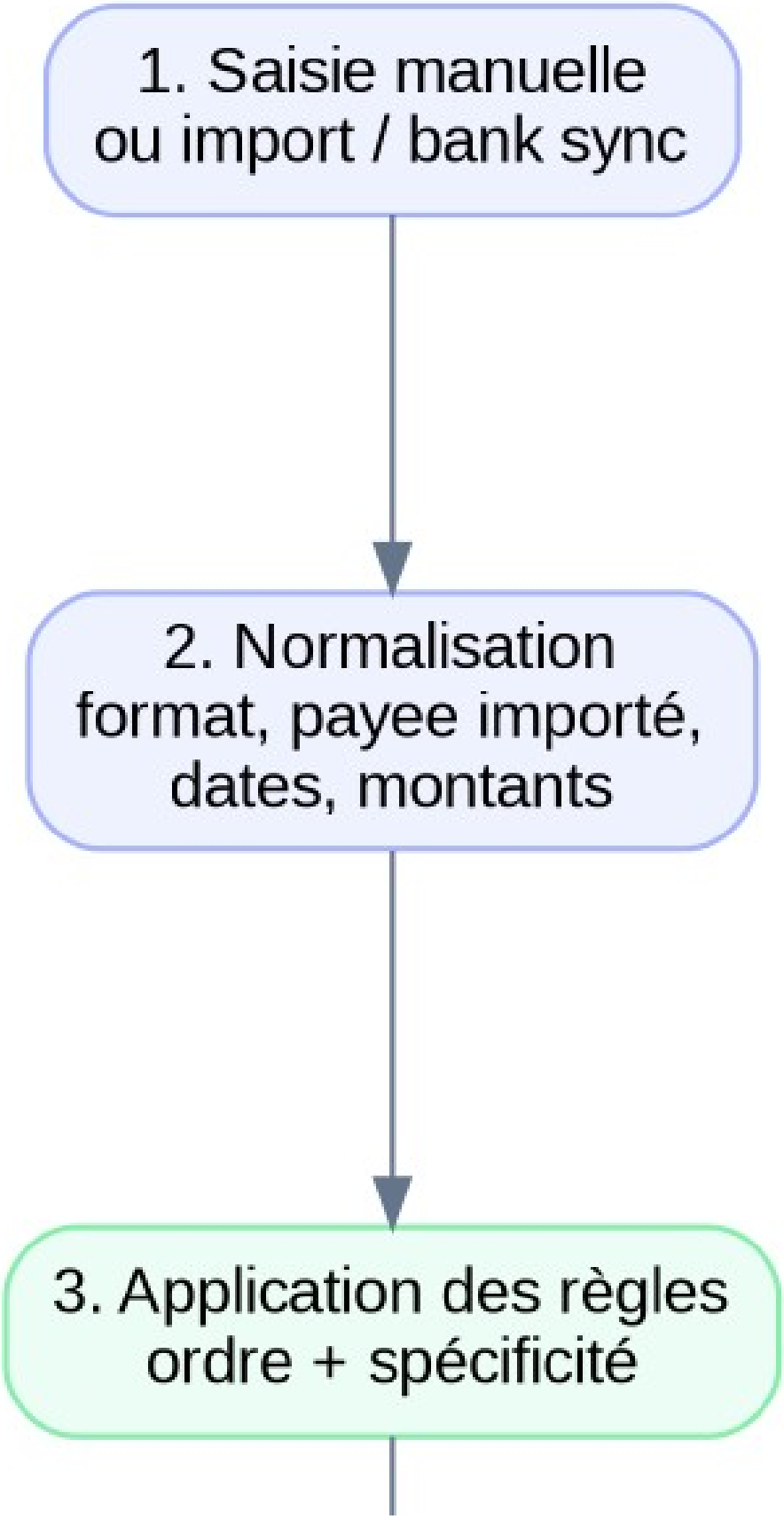

1. Saisie manuelle
ou import / bank sync
2. Normalisation
format, payee importé,
dates, montants
3. Application des règles
ordre + spécificité

Figure 4 - Cycle de vie générique d'une transaction

1. Entrée : saisie manuelle, import de fichier, bank sync ou génération depuis un schedule.
2. Normalisation : adaptation des champs entrants, formats de dates, montants, nettoyage du libellé importé.
3. Règles : exécution ordonnée des conditions/actions, résolution éventuelle du payee et de la catégorie.
4. Résolution métier : transfert, split, rattachement à un schedule, apprentissage de catégorie.
5. Persistance : transaction-add, transaction-update ou transactions-batch-update.
6. Effets dérivés : mise à jour des balances, du budget, des suggestions et des rapports.
7. Synchronisation : publication des changements si un sync-server est configuré.

7. Parcours détaillés

7.1 Création d'un compte

Le parcours de création propose soit un compte local, soit un rattachement à un provider de synchronisation bancaire. Les informations demandées côté UI sont le nom, le statut on/off budget et le solde courant. Le compte est ensuite persisté par le cœur métier et visible dans la sidebar.

Étape	Entrée / décision	Effet métier
1	Choix compte local ou bank sync	Oriente le flux d'initialisation
2	Nom du compte	Crée l'identité visible du registre
3	On budget / off budget	Détermine l'impact sur le budget
4	Solde initial	Initialise balance et historique
5	Création côté cœur	Retourne un id et met à jour la navigation

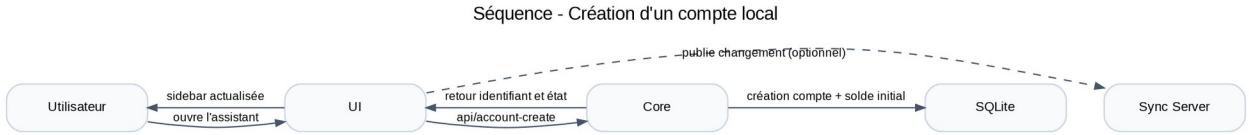


Figure 5 - Séquence de création d'un compte local

7.2 Ajout manuel d'une transaction

Une transaction manuelle peut être saisie dans un registre de compte, dans All Accounts ou dans certaines vues liées aux catégories. L'interface propose autocomplete payee/catégorie, puis le cœur métier persiste la transaction et recalcule les effets budgétaires.

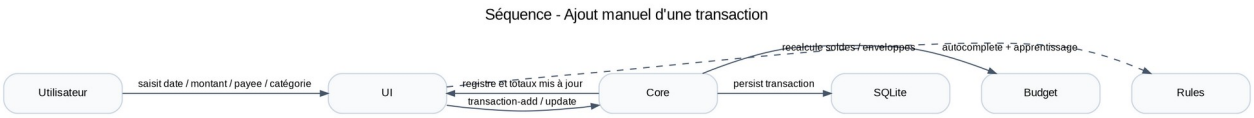


Figure 6 - Séquence d'ajout manuel d'une transaction

- Les montants sont manipulés en cents entiers dans les APIs/CLI.
- Les champs payee et catégorie peuvent être déduits ou proposés selon l'historique.
- La persistance peut déclencher apprentissage implicite et mises à jour d'agrégats.

7.3 Import de transactions et application des règles

L'import peut provenir d'un fichier ou d'un provider bancaire. Les transactions importées traversent une chaîne de normalisation puis sont comparées aux règles existantes. Le moteur peut reconnaître un payee, appliquer une catégorie par défaut, forcer un compte de transfert ou enrichir les notes.

Figure 7 - Séquence d'import et de traitement par règles

- Si un payee exact existe, il est prioritaire.
- Sinon le système peut appliquer des règles de type contains/exact sur le payee importé.
- Les règles s'exécutent dans l'ordre affiché ; la plus spécifique gagne en cas de conflit de champ.
- Le batch update est le point de persistance typique pour les imports.

7.4 Gestion d'un transfert

Un transfert n'est pas un type d'objet autonome : il est modélisé comme un payee spécial représentant un autre compte. Lorsqu'un transfert est saisi, le moteur crée la transaction source et la transaction miroir dans le compte cible.

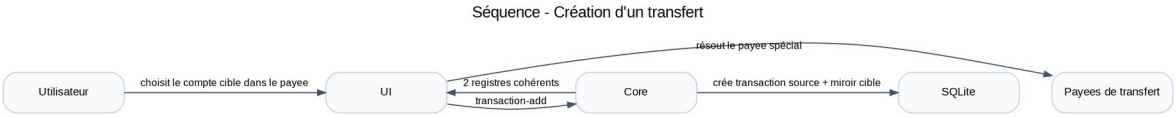


Figure 8 - Séquence de création d'un transfert

- Cette modélisation permet d'utiliser les règles aussi sur les transferts.
- Les imports synchronisés sur les deux comptes nécessitent un paramétrage de règles prudent pour éviter les doublons.
- Les transactions enfants/splits peuvent compliquer la relation miroir.

7.5 Rapprochement bancaire

Le rapprochement compare le relevé bancaire et le registre Actual. L'utilisateur saisit une balance cible ; le système calcule la différence avec le cleared total. Le marquage progressif des opérations en cleared réduit l'écart jusqu'à convergence.

Figure 9 - Séquence de rapprochement bancaire

- Le lock icon ouvre le module de rapprochement.
- Le système expose cleared total et uncleared total dans l'en-tête du compte.
- La réconciliation met à jour des informations de balance et de date du dernier rapprochement.

7.6 Scheduled transactions

Le moteur de schedules couvre les récurrences simples et avancées. Une occurrence peut être postée automatiquement ou proposée pour validation. Un schedule peut également être relié à des transactions déjà présentes.

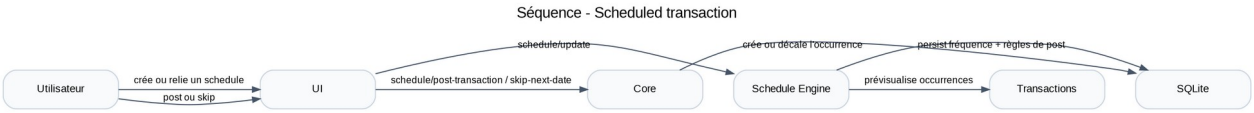


Figure 10 - Séquence de gestion d'un schedule

- Le système sait poster, sauter la prochaine date, ou marquer un schedule comme complété.
- Les schedules sont visibles aussi comme transactions preview dans certaines listes.
- Le lien entre une transaction et son schedule est important pour éviter les doublons.

7.7 Synchronisation multi-appareils

Actual adopte une logique local-first : chaque appareil manipule une copie locale, puis synchronise les changements vers un sync-server. L'utilisateur garde un usage fonctionnel même hors connexion.

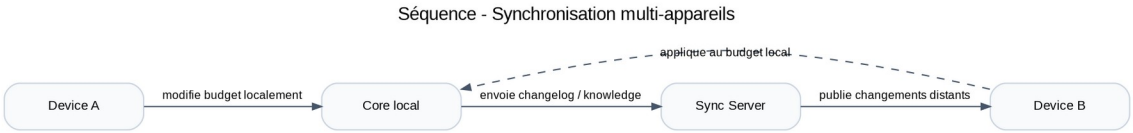


Figure 11 - Séquence de synchronisation multi-appareils

- Le serveur stocke et redistribue des changements ; il n'est pas l'unique lieu d'exécution métier.
- Le fichier peut être chiffré de bout en bout, avec les contraintes de récupération que cela implique.
- Le budget est sélectionné/téléchargé depuis le serveur puis synchronisé en tâche de fond.

8. Gestion détaillée des objets métier

8.1 Comptes

- Un compte peut être checking, credit card, loan, investment ou autre type exposé par l'UI.
- Le booléen onBudget pilote si les transactions de ce compte alimentent le budget.
- Un compte hidden/closed reste historisé mais n'est plus dans le flux actif standard.
- Les balances et le rapprochement sont rattachés au compte.

8.2 Budget et catégories

- Le budget manipule des allocations mensuelles par sous-catégorie.
- Les catégories sont organisées en groupes ; l'un des groupes concerne l'income et possède des règles particulières.
- Le système gère fusion, suppression, masquage et report de soldes.
- Les dépassements peuvent être traités selon des règles d'overspending handling.

8.3 Transactions, splits, merge et export

- Les handlers observés incluent transaction-add, transaction-update, transaction-delete, transaction-move, transactions-batch-update et transactions-merge.
- L'export CSV reconstruit des libellés lisibles à partir des noms de comptes, catégories et payees.
- Le système distingue parent/child pour les splits et expose un mode d'export compatible avec cette structure.

8.4 Payees

- Le payee sert de contrepartie normalisée et d'ancre d'automatisation.
- Les payees peuvent être fusionnés, renommés, marqués favoris, enrichis de règles et d'une catégorie par défaut.
- Les comptes de transfert existent aussi sous forme de payees spéciaux.

8.5 Rules

- Une règle contient des conditions et des actions sérialisées.
- Le moteur valide, classe et exécute les règles sur les flux d'import ou d'édition concernés.
- Le système crée aussi des règles implicitement à partir du comportement utilisateur.

8.6 Reports et AQL

- Les rapports standard couvrent notamment net worth et cash flow.
- Les custom reports s'appuient sur un moteur de requête capable de compiler et d'exécuter des requêtes AQL.
- L'API de query permet des agrégations puissantes sans contourner le modèle métier.

9. Points d'attention pour une future évolution

Sujet	Pourquoi c'est sensible	Question d'architecture à se

		poser
Transactions	Objet pivot avec nombreux effets secondaires	Ajout d'état métier ou enrichissement sans casser imports/sync ?
Rules	Automatisation implicite + explicite	Comment rendre la règle compréhensible et testable ?
Payees	Fort couplage avec imports et UX	Peut-on renforcer la normalisation sans perturber l'apprentissage ?
Schedules	Risque de doublons et de conflits avec imports	Quelle stratégie d'idempotence ?
Sync	Local-first + chiffrement + conflits	Quel impact d'un nouveau champ sur le protocole de changement ?
Budget	Calculs sensibles à la temporalité et aux dépassements	Comment préserver la cohérence des rollovers ?
Reports	Forte attente utilisateur sur la fidélité des chiffres	Quel niveau de test de non-régression analytique ?

10. Livrables graphiques inclus

Fichier	Type	Usage
01_architecture_globale.(png/svg/dot)	Diagramme	Vue d'ensemble de l'architecture
02_runtime_web_vs_electron.(png/svg/dot)	Diagramme	Comparaison des runtimes
03_classes_fonctionnelles.(png/svg/dot)	Diagramme	Vue domaine / objets métier
04_cycle_vie_transaction.(png/svg/dot)	Diagramme	Chaîne de traitement transaction
05 à 11_sequence_*.png/svg/dot	Diagrammes de séquence	Parcours métier majeurs
*.drawio	Source éditable	Réédition dans diagrams.net / draw.io

11. Conclusion

Actual présente une architecture cohérente pour un produit local-first : l'interface reste légère, le cœur métier est concentré, et la persistance locale joue un rôle central. La meilleure stratégie d'évolution consiste généralement à traiter Transaction, Payee, Rule, Schedule et Budget comme un

noyau fortement lié, tout en maintenant une frontière claire entre UI, logique métier et synchronisation.

Le dossier joint doit servir de base de lecture. Pour une phase suivante, il serait pertinent de produire en complément un document d’impact ciblé par chantier (par exemple : refonte budget, moteur de catégorisation intelligente, nouveau workflow d’import, ou exposition d’API).